

VivoGuard AI

System Documentation & Operator Handbook

AI-powered video surveillance platform for Vivo Fashion Group's 26 retail stores across Kenya, Uganda, and Rwanda. Built on FastAPI + Celery + PostgreSQL + Redis + MinIO with YOLOv8 inference.

Audience	Store managers, regional operators, head-office IT
Stack	Docker Compose · Python 3.11 · React 18 · Postgres 16 · Redis 7
Branch	claude/vivoguard-ai-platform-YE5iF
Repo path	/home/user/VivoGuard-AI
First boot	docker compose up -d --build
Login	Browser → http://<host>/ (bootstrap admin from env)

Contents

- 1. System overview & architecture
- 2. Container topology & ports
- 3. Database schema — key tables
- 4. API surface — endpoints by area
- 5. Detector catalogue
- 6. Camera transport & tunnel modes
- 7. Alerts pipeline & severity
- 8. Scheduled tasks (Celery beat)
- 9. Configuration & environment variables
- 10. Deployment commands
- 11. Troubleshooting playbook (cameras / API / DB / detectors)
- 12. Common one-liners (operator cheat sheet)
- 13. Glossary

1. System overview & architecture

VivoGuard AI is a single-tenant deployment that runs entirely on the operator's hardware. There are no external SaaS dependencies for the core surveillance pipeline; only Twilio (for SMS / WhatsApp delivery) and SMTP (for email) are optional integrations.

Data flow

- 1. Streamer container** opens one connection per camera (RTSP / TCP, RTSP / HTTP-tunnel, or HTTP snapshot polling) and publishes each JPEG frame to Redis under `vg:frame:{camera_id}`.
- 2. Worker (inference) container** consumes those frames, runs YOLOv8 (CPU by default; GPU when configured), executes the registered detectors, and writes **DetectionEvent** + **MetricSnapshot** rows to Postgres. Operator-visible events also create **Alert** rows.
- 3. API container** (FastAPI) exposes the REST surface, serves the React bundle via nginx (frontend container), and publishes alert events on the `vg:pub:alerts` Redis channel.
- 4. Alert engine** (background asyncio task inside API) subscribes to that channel, deduplicates, and fans out to SMTP / Twilio SMS / WhatsApp / webhook notifiers in parallel.
- 5. Celery beat** drives periodic work: report dispatch, staff classifier, heatmap archive, morning briefings, scheduled report generation.

Why this shape

Streamer vs worker is split so the heavy FFmpeg I/O lives next to the cameras and the ML inference can scale independently. The streamer image is a thin layer on top of the API image (no torch / no opencv); the worker image is the API image plus torch + ultralytics. That keeps the API + streamer footprint under 350 MB.

Tiered transport for cameras

Stores' routers do not consistently forward port 554. The platform auto-negotiates per camera at first probe:

Tier 1 — Direct RTSP over TCP on the configured port (usually 554, sometimes 7000 on Dahua-Vivo NVRs).

Tier 2 — RTSP-over-HTTP tunnel using FFmpeg's `-rtsp_transport http` flag on an HTTP admin port. Full video preserved.

Tier 3 — HTTP snapshot polling using the Dahua `/cgi-bin/snapshot.cgi` or Hikvision `/ISAPI/Streaming/channels/N01/picture` or generic ONVIF media URL on an HTTP port. Lower frame rate, but works wherever a browser does.

2. Container topology & ports

Service	Image	Internal port	External port	Purpose
api	vivoguard/api	8000	via nginx	FastAPI REST + WebSocket
worker	vivoguard/worker	—	—	Celery worker + beat (inference)
streamer	vivoguard/streamer	—	—	FFmpeg / HTTP-snapshot fleet
frontend	vivoguard/frontend	80	via nginx	React bundle (nginx static)
postgres	postgres:16	5432	—	Time-series + entity store
redis	redis:7	6379	—	Pub/sub + frame cache + locks
minio	minio/minio	9000	—	S3-compatible blob store (clips, thumbnails, models)
nginx	deploy/nginx	80	80	Reverse proxy → api + frontend

Horizontal scaling — streamer shards

For fleets above ~60 cameras, the streamer can be split across multiple containers using the bundled overlay:

```
docker compose -f docker-compose.yml \
-f docker-compose.scale.yml \
up -d --build
```

Each replica claims cameras WHERE id % STREAMER_SHARD_COUNT == STREAMER_SHARD_INDEX. Every camera lands on exactly one replica; no coordination needed.

Network

All containers share the vivoguard bridge network. The host exposes only nginx on port 80 by default. Camera traffic stays inside the LAN / VPN — nothing camera-facing is exposed publicly.

3. Database schema — key tables

Selected tables most operators interact with via the SQL CLI. Full schema lives in `backend/app/models/` and is materialised by Alembic migrations 0001 → 0011.

stores

Column	Type	Notes
id	int PK	—
name	varchar(128)	unique, indexed
code	varchar(32) NULL	unique when set
country	varchar(64)	Kenya / Uganda / Rwanda / Other
timezone	varchar(64)	Africa/Nairobi default
business_hours_json	jsonb	per-weekday open windows
capacity	int NULL	occupancy-alert threshold input
manager_name	varchar	WhatsApp briefing personalisation
manager_phone	varchar	WhatsApp briefing recipient
default_rtsp_port	int NULL	inherited by Add Camera wizard

cameras

Column	Type	Notes
id, name, brand	—	brand ∈ dahua hikvision uniview onvif generic
host, public_ip	varchar	host = local IP or DDNS, public_ip set for WAN
rtsp_port, http_port	int	rtsp_port often 7000 on Vivo Dahua NVRs
channel_number	int NULL	NVR channel index
password_encrypted	text	Fernet-encrypted credentials
transport	varchar(16)	rtsp http_snapshot
rtsp_transport	varchar(8)	tcp http udp — FFmpeg flag
snapshot_url_override	text NULL	set by auto-negotiator on non-Dahua tunnels
store_id	FK stores.id NULL	set by Quick Add / store-first onboarding
ai_enabled, inference_fps	bool/int	—
status, last_seen_at, last_error	—	informational; Redis health is source of truth

Other key tables

metric_snapshots	Time-series — occupancy, queue_length, queue_wait_seconds, dwell_seconds, staff_present_pct, visitor_count_in, etc. Indexed by (store_id, metric_type, period_start).
detection_events	Raw inference output — one row per detection. detection_type + zone_id + confidence + bbox_json.

alerts	Operator-visible incidents. status ∈ new / confirmed / dismissed / escalated. Joined to detection_events via event_id.
visitor_tracks	Daily-dedup'd person tracks. (store_id, day, track_signature) unique.
customer_journeys	Per-track zone sequence: zone_sequence_json holds the visit timeline.
staff_tracks	Output of the 10-min staff classifier — counter_seconds + classified_as ∈ staff / customer.
scheduled_reports	Recurring report definitions — cadence + time_of_day + day_of_week + recipients.
zones	Per-camera polygon / tripwire definitions. detection_types_json drives which detectors run.
heatmap_snapshots	30-day archive of per-day heatmap PNGs.

4. API surface — endpoints by area

All endpoints are mounted under `/api` when behind nginx. Auth is JWT bearer on every route except `/auth/login` and `/healthz`.

Auth

Method	Path	Purpose
POST	<code>/auth/login</code>	Email + password → access + refresh JWT
POST	<code>/auth/refresh</code>	Refresh → new access token
GET	<code>/auth/me</code>	Current user

Cameras

Method	Path	Purpose
GET	<code>/cameras</code>	List
POST	<code>/cameras/add</code>	Create (single)
POST	<code>/nvr/quick-add</code>	Bulk-add N channels of one NVR
PATCH	<code>/cameras/{id}</code>	Update fields
DELETE	<code>/cameras/{id}</code>	Remove
GET	<code>/cameras/{id}/snapshot</code>	Latest JPEG (Redis cache → ffmpeg fallback)
POST	<code>/cameras/test-rtsp-ports</code>	Smart port probe for new cameras
POST	<code>/cameras/intelligent-probe</code>	Multi-port + multi-scheme HTTP snapshot probe
POST	<code>/cameras/{id}/try-alternate-ports</code>	Per-camera RTSP / tunnel port iteration
POST	<code>/cameras/{id}/transport-diagnose</code>	Full probe report (per port + URL)
POST	<code>/cameras/auto-failover</code>	Chain-wide auto switch RTSP → HTTP-snapshot
POST	<code>/cameras/bulk-update-port</code>	Set <code>rtsp_port</code> on every camera at a host
GET	<code>/cameras/brand-defaults</code>	Per-brand default ports for the wizard
GET	<code>/cameras/cv-quality</code>	Average detection confidence per camera (24h)

Analytics — store dashboard

Method	Path	Purpose
GET	<code>/analytics/store/{id}/live</code>	Right Now + Today So Far tiles
GET	<code>/analytics/store/{id}/hourly</code>	Hourly footfall + auto-insights
GET	<code>/analytics/store/{id}/week-summary</code>	7-day bars + detector activity
GET	<code>/analytics/store/{id}/behaviour</code>	Customer journey paths (staff excluded)
GET	<code>/analytics/store/{id}/staff-timeline</code>	Per-minute staffed/unstaffed timeline
GET	<code>/analytics/store/{id}/zone-performance</code>	Aisle engagement ranking
GET	<code>/analytics/store/{id}/scorecard</code>	Today vs yesterday RAG table
GET	<code>/analytics/store/{id}/anomalies</code>	30-day z-score per (weekday, hour)
GET	<code>/analytics/store/{id}/staffing-prediction</code>	Recommended staff per (weekday, hour)

Method	Path	Purpose
GET	/analytics/store/{id}/health-score	0-100 composite + components
GET	/analytics/store/{id}/loss-prevention	7-day LP rollup + hotspots
GET	/analytics/store/{id}/behaviour-trends	Weekly trends + conversion funnel
GET	/analytics/store/{id}/visitor-intelligence	Staff exclusion + dedup
GET	/analytics/dashboard/multi	Chain dashboard + RAG per store
GET	/analytics/chain/health-leaderboard	Ranked health scores
GET	/analytics/heatmap/{cam}/image	PNG (alpha + window=hour day week)
GET	/analytics/report.pdf	Ad-hoc PDF (?store_id&since&until)
GET	/analytics/report.csv	Ad-hoc CSV

Alerts & reports

Method	Path	Purpose
GET	/alerts	List (?store_id, ?status, ?detection_type, ...)
POST	/alerts/{id}/resolve	Mark resolved (everyday action)
POST	/alerts/{id}/confirm	True positive → feeds ML training
POST	/alerts/{id}/dismiss	False positive → feeds ML training
POST	/alerts/{id}/note	Append investigation note
GET	/alerts/{id}/snapshot	JPEG (event thumb → Redis fallback)
GET	/reports/scheduled	List schedules
POST	/reports/scheduled	Create schedule
DELETE	/reports/scheduled/{id}	Delete
POST	/reports/scheduled/{id}/run-now	Queue immediate dispatch

System & diagnostics

Method	Path	Purpose
GET	/healthz	Liveness probe (no auth)
GET	/system/health	Disk / GPU / camera summary
GET	/system/schema-check	Compare ORM expectations vs live DB
GET	/system/cameras/{id}/stream-health	Per-camera fps + last frame + errors
GET	/detectors/catalog	All detector types + status badges

5. Detector catalogue

Every detector listed here is implemented and runs on CPU. Detectors marked *training required* need a custom YOLO model — until trained, they're inactive even when enabled.

Detector	Plain-English name	Default threshold	Alert?
occupancy	People in Store	≥80% of capacity	✓
queue	Checkout Queue	>3 people waiting	✓
dwel	Product Interest Time	>5 min in aisle	✓
staff_present	Counter Staffing	Unstaffed >5 min	✓
intrusion	After-hours Security	Outside business hrs	✓ critical
fight	Incident Detection	Aggressive motion	✓ critical
fall	Fall Detection	Aspect ratio change	✓ critical
weapon	Weapon Detected	Custom model	✓ critical · training required
fire / smoke	Fire / Smoke	Custom model	✓ critical · training required
shrinkage	Loss Prevention	Behaviour pattern	✓ critical
trespass	Restricted Area	Zone-gated	✓
loitering	Loitering Alert	>10 min single area	✓
tailgating	Tailgate Alert	Multi-person entry	✓
abandoned_object	Abandoned Item	Object > 60 s still	✓
crowd	Crowd Alert	>8 in zone	✓
shutter	Store Open/Close	Out-of-hours change	✓
stockroom_access	Stockroom Access	Off-roster entry	✓
passersby	Window Passersby	Sidewalk counter	—
window_engagement	Window Engagement	>5 s window dwell	—
unique_visitor	Customer Count	—	— (metric)
entry_exit	Entry / Exit	—	— (metric)
heatmap	Busiest Areas	—	— (metric)
customer_journey	Customer Journey	—	— (metric)
demographic	Visitor Demographics	—	— (privacy)
lpr	Plate Recognition	Vehicle bbox + OCR	— (metric)
face	Face Detection	Custom model	training required
uniform_compliance	Uniform Compliance	Custom model	training required

Default state

As of migration 0011, opening the AI Settings page for any camera shows every detector pre-checked with the thresholds above. Operators just need to click Save once to persist the row; the inference worker picks it up within ~10 s.

6. Camera transport & tunnel modes

Probe order (auto-failover & Test RTSP)

#	Step	Latency budget	Outcome
1	TCP probe of (host, rtsp_port)	2 s	If reachable → keep RTSP
2	RTSP tunnel: ffprobe -rtsp_transport http on 7000, 80, 800, 8000, 8080	8 s/port	First port that negotiates wins
3	HTTP snapshot: GET /cgi-bin/snapshot.cgi (Dahua) and equivalents (Hikvision ISAPI, ONVIF, Axis)	4 s/URL	First URL that returns JPEG magic wins
4	Fail	—	Camera stays RTSP, tile shows Diagnose button

Per-brand URL templates

Brand	Template
Dahua	rtsp://USER:PASS@HOST:PORT/cam/realmonitor?channel=N&subtype=0
Hikvision	rtsp://USER:PASS@HOST:PORT/Streaming/Channels/N01
Uniview	rtsp://USER:PASS@HOST:PORT/unicast/cN/s1/live

When tunnel mode wins, the URL is the same — the only difference is FFmpeg negotiates over the HTTP port instead of opening a separate 554 connection. Operators don't need to modify any URL.

Snapshot probe URL list

Dahua-CGI	http(s)://host:port/cgi-bin/snapshot.cgi?channel=N
Hikvision ISAPI	http(s)://host:port/ISAPI/Streaming/channels/N01/picture
Hikvision legacy	http(s)://host:port/Streaming/channels/N01/picture
ONVIF snap	http(s)://host:port/onvif/media_service/snapshot?ProfileToken=Profile_N
Axis / Vivotek	http(s)://host:port/axis-cgi/jpg/image.cgi?camera=N

HTTPS variants attempted automatically with TLS verification off (self-signed certs on NVRs are normal).

7. Alerts pipeline & severity

Flow

1. Detector emits a `DetectionEvent`. Most events also create an `Alert` row (`status=new`). Silent metric detectors — `heatmap`, `customer_journey`, `unique_visitor`, `entry_exit`, `demographic`, `occupancy_metrics` — never create alerts.
2. Inference worker publishes the event to Redis channel `vg:pub:alerts`.
3. The alert engine (async task inside the API container) subscribes, deduplicates per (`camera_id`, `detection_type`, `zone_id`) within 30 s (5 s for high-priority), hydrates camera + zone names, and fans out to every enabled notifier in parallel.
4. The `/alerts` page and the per-store Alerts & Incidents section render server-rendered title + body + severity + snapshot URL — the frontend stops translating `detection_type` strings.

Severity buckets

Bucket	Types
■ critical (red)	fight, intrusion, weapon, weapon_brandished, fall, fire, smoke, shrinkage
■ warning (amber)	queue, queue_length, crowd, trespass, loitering, shutter, abandoned_object, tailgating
■ info (blue)	staff_present, occupancy, entry_exit, dwell, passersby

Notifiers

Channel	Env vars	Gated on
SMTP email	SMTP_HOST, SMTP_USER, SMTP_PASSWORD, SMTP_FROM	Any event
Twilio SMS	TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_FROM_NUMBER	Any event
WhatsApp	TWILIO_WHATSAPP_FROM, WHATSAPP_TO, WHATSAPP_PRIORITY_ONLY	<code>extra.priority == 'high'</code> (default)
Webhook	WEBHOOK_URL, WEBHOOK_AUTH_HEADER	Any event

8. Scheduled tasks (Celery beat)

All beat ticks run inside the worker container. The worker is started with the `-B` flag so beat and worker share a process — no separate beat container.

Task	Schedule	What it does
<code>inference.supervise_all</code>	30 s	Spawns a <code>run_camera_inference</code> task per <code>ai_enabled</code> camera
<code>maintenance.refresh_ddns</code>	5 min	Resolves DDNS hostnames for WAN cameras
<code>reports.dispatch_due</code>	5 min	Walks <code>scheduled_reports</code> ; fires any whose <code>time_of_day + day_of_week</code> match (store-local)
<code>heatmap.snapshot_all</code>	≈ 24 h	Daily heatmap PNG archive (30-day rolling retention)
<code>staff_classifier.classify_today</code>	10 min	Flags tracks with >10 min counter-zone dwell as staff
<code>briefings.daily_fire_due</code>	5 min	Sends 08:00 store-local WhatsApp briefing to <code>manager_phone</code>
<code>briefings.weekly_fire_due</code>	5 min	Sends Monday 07:00 chain summary to <code>WEEKLY_BRIEFING_TO</code>

Inspecting beat

```
# What's registered?
docker compose exec worker celery -A app.tasks.celery_app inspect registered

# What's running right now?
docker compose exec worker celery -A app.tasks.celery_app inspect active

# Trigger a task immediately for testing.
docker compose exec worker python -c "
from app.tasks.briefings import daily_fire_due
daily_fire_due.delay()"

```

9. Configuration & environment variables

Defined in backend/app/config.py; operators set them via .env or docker-compose.override.yml.

Required

JWT_SECRET	32+ char random string
BOOTSTRAP_ADMIN_EMAIL	First admin's email — created on first boot
BOOTSTRAP_ADMIN_PASSWORD	Initial password (changeable in UI after login)
POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_DB	Database creds
CREDENTIALS_FERNET_KEY	Fernet key for encrypting camera passwords (44-char base64)
S3_ACCESS_KEY / S3_SECRET_KEY	MinIO creds (default: minioadmin / minioadmin)

Optional — alert delivery

SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASSWORD / SMTP_FROM	Email
TWILIO_ACCOUNT_SID / TWILIO_AUTH_TOKEN	Twilio creds
TWILIO_FROM_NUMBER	SMS sender, +
TWILIO_WHATSAPP_FROM	WhatsApp sender, whatsapp:+
WHATSAPP_TO	Alert recipients, comma-separated whatsapp:+
WHATSAPP_PRIORITY_ONLY	true (default) → only high-priority alerts go to WhatsApp
WEEKLY_BRIEFING_TO	Chain weekly summary recipients (Monday 07:00)
WEBHOOK_URL / WEBHOOK_AUTH_HEADER	Generic webhook destination

Optional — operational

STREAMER_SHARD_COUNT / STREAMER_SHARD_INDEX	Set to scale streamers across containers
STREAMER_POLL_INTERVAL	How often the reconcile loop runs (default 5 s)

DEFAULT_MODEL	YOLOv8 weights path (default yolov8n.pt)
INFERENCE_FPS_DEFAULT	Per-camera FPS when not set on the row
USE_GPU / CUDA_VISIBLE_DEVICES	Toggle GPU inference

10. Deployment commands

First boot (fresh host)

```
git clone <repo>
cd VivoGuard-AI
cp .env.example .env # then edit JWT_SECRET, bootstrap admin, etc.
docker compose up -d --build
# Wait ~60s for postgres + alembic to settle.
docker compose logs --tail=20 api | grep 'alembic upgrade head'
```

Standard update

```
git pull
docker compose build api worker frontend
docker compose up -d --force-recreate api worker frontend
```

Force a clean rebuild (after weird cache behaviour)

```
git pull
docker compose build --no-cache api worker frontend streamer
docker compose up -d --force-recreate api worker frontend streamer
# In the browser, hard-refresh: Ctrl+Shift+R
```

Scaled streamer (60+ cameras)

```
docker compose -f docker-compose.yml \
-f docker-compose.scale.yml \
up -d --build
# Verify all three replicas:
docker compose logs streamer | grep 'shard 0 of 3'
docker compose logs streamer-2 | grep 'shard 1 of 3'
docker compose logs streamer-3 | grep 'shard 2 of 3'
```

Database backup & restore

```
# Backup (run nightly via cron or Task Scheduler)
docker compose exec -T postgres pg_dump -U $POSTGRES_USER \
-d $POSTGRES_DB --no-owner --no-acl \
| gzip > backups/vivoguard-$(date +%Y%m%d).sql.gz

# Restore
gunzip -c backups/vivoguard-20260520.sql.gz \
| docker compose exec -T postgres psql -U $POSTGRES_USER -d $POSTGRES_DB
```

11. Troubleshooting playbook

Symptom-first guide. Run each command in sequence; the first that returns a useful answer points to the fault.

11.1 Site is completely down (browser shows nothing / connection refused)

```
docker compose ps
# Look for any service NOT in 'Up' state. Common offenders: api, frontend, postgres.

docker compose logs --tail=80 api
# Look for: alembic errors, 'Address already in use', bcrypt failures.

docker compose logs --tail=40 frontend
# nginx errors here mean the static build failed.

docker compose restart api frontend
# Usually unblocks transient issues.

# If still down — rebuild without cache:
docker compose build --no-cache api frontend
docker compose up -d --force-recreate api frontend
```

11.2 API returns 500 on every request

```
docker compose logs --tail=120 api | grep -E 'Traceback|ERROR'

# Schema drift? (alembic version vs actual columns)
curl -s http://localhost/api/system/schema-check \
-H "Authorization: Bearer $env:VG_TOKEN" | python -m json.tool

# If 'missing' is non-empty:
docker compose exec api alembic upgrade head
```

11.3 Camera shows 'Stream paused — Connection refused' on Live View

```
# 1. Diagnose from the tile: click '■ Diagnose & auto-fix'.
# The modal lists every port and URL that was tried.

# 2. From CLI, force a TCP probe from inside the api container:
docker compose exec api python -c "
import asyncio, socket
from app.utils.network import tcp_open
print(asyncio.run(tcp_open('197.155.67.50', 554, timeout=3)))"

# 3. Try the alternate-port endpoint directly:
curl -X POST -H "Authorization: Bearer $env:VG_TOKEN" \
http://localhost/api/cameras/14/try-alternate-ports

# 4. If RTSP IS unreachable but HTTP works on the NVR's web UI,
# confirm the camera's rtsp_transport is set to 'http':
docker compose exec postgres psql -U $POSTGRES_USER -d $POSTGRES_DB \
-c "SELECT id,name,host,rtsp_port,rtsp_transport,transport FROM cameras WHERE id=14;"
```

11.4 All cameras dark (streamer not producing frames)

```
docker compose ps streamer
docker compose logs --tail=80 streamer

# Migration drift kills the streamer reconcile loop. Force the
# migration explicitly:
docker compose exec api alembic upgrade head
```

```
docker compose restart streamer

# If the streamer image is stale (rebuilt api but not streamer):
docker compose build --no-cache streamer
docker compose up -d --force-recreate streamer

# Confirm reconcile is producing specs:
docker compose logs --tail=20 streamer | grep 'reconcile'
# Expect lines like: 'reconcile: 21 desired cameras, 21 running workers'
```

11.5 Dashboard shows 0 for queue / staff / dwell

```
# Confirm rows exist for the store_id:
docker compose exec postgres psql -U $POSTGRES_USER -d $POSTGRES_DB \
-c "SELECT metric_type, COUNT(*), ROUND(AVG(value)::numeric, 2)
FROM metric_snapshots
WHERE store_id = 2
AND period_start > now() - interval '12 hours'
GROUP BY metric_type;"

# If rows exist but the API reads 0, the camera_id ↔ store_id link is broken.
# As of commit 82b01c4 the analytics filters use
# (MetricSnapshot.store_id == store_id) OR camera_id IN cam_ids
# so this should self-heal once the api container is on that commit.
```

11.6 Detectors are configured but no alerts arrive

```
# 1. Confirm the inference worker is processing frames:
docker compose logs --tail=40 worker | grep -i detector

# 2. Are detection_events being written?
docker compose exec postgres psql -U $POSTGRES_USER -d $POSTGRES_DB \
-c "SELECT detection_type, COUNT(*) FROM detection_events
WHERE timestamp > now() - interval '1 hour'
GROUP BY detection_type;"

# 3. Are alerts being created?
docker compose exec postgres psql -U $POSTGRES_USER -d $POSTGRES_DB \
-c "SELECT status, COUNT(*) FROM alerts
WHERE created_at > now() - interval '1 hour' GROUP BY status;"

# 4. Confirm the alert engine is alive:
docker compose logs --tail=40 api | grep 'alert engine'
# Expect: 'alert engine: subscribed to vg:pub:alerts (notifiers=...)'
```

11.7 WhatsApp / email notifications not arriving

```
# Twilio: confirm env vars are set on the worker.
docker compose exec worker env | grep -E 'TWILIO|WHATSAPP|SMTP'

# Fire a test briefing manually:
docker compose exec worker python -c "
from app.tasks.briefings import daily_fire_due
daily_fire_due.delay()"

# Watch logs:
docker compose logs -f worker | grep -iE 'briefing|whatsapp|twilio'
```

11.8 Login fails with 401 / 'invalid credentials' immediately after deploy

```
# The bcrypt build smoke test catches most cases at build time.
# If it slipped through:
docker compose exec api python -c "
```



```
import bcrypt
h = bcrypt.hashpw(b'test', bcrypt.gensalt(rounds=4))
print(bcrypt.checkpw(b'test', h))"
# Should print True. If it raises, rebuild api:
docker compose build --no-cache api
docker compose up -d --force-recreate api
```

11.9 Reset the admin password

```
docker compose exec api python -c "
from app.database import SessionLocal
from app.models import User
from app.auth.security import hash_password
with SessionLocal() as s:
    u = s.query(User).filter(User.email == 'admin@example.com').first()
    u.password_hash = hash_password('NEW-PASSWORD-HERE')
    s.commit()
print('reset:', u.email)"
```

12. Common one-liners — operator cheat sheet

Health

```
# All container statuses
docker compose ps

# Tail every service
docker compose logs -f --tail=40

# API liveness (no auth required)
curl http://localhost/api/healthz

# Detector activity for a store
curl -s -H "Authorization: Bearer $env:VG_TOKEN" \
http://localhost/api/analytics/store/2/week-summary | python -m json.tool
```

Cameras

```
# List, with online status
curl -s -H "Authorization: Bearer $env:VG_TOKEN" \
http://localhost/api/cameras | python -m json.tool

# Force auto-failover across the fleet
curl -X POST -H "Authorization: Bearer $env:VG_TOKEN" \
http://localhost/api/cameras/auto-failover

# Per-camera per-port diagnostic
curl -X POST -H "Authorization: Bearer $env:VG_TOKEN" \
http://localhost/api/cameras/14/transport-diagnose?fresh=true
```

Database

```
# Open a psql shell
docker compose exec postgres psql -U $POSTGRES_USER $POSTGRES_DB

# Recent alerts
SELECT a.id, e.detection_type, c.name, a.status, a.created_at
FROM alerts a JOIN detection_events e ON a.event_id = e.id
LEFT JOIN cameras c ON e.camera_id = c.id
WHERE a.created_at > now() - interval '1 hour'
ORDER BY a.created_at DESC LIMIT 20;

# Detector event counts today
SELECT detection_type, COUNT(*)
FROM detection_events
WHERE timestamp > date_trunc('day', now())
GROUP BY 1 ORDER BY 2 DESC;
```

Redis

```
# Open a redis-cli shell
docker compose exec redis redis-cli

# Current frame for a camera (size in bytes)
STRLEN vg:frame:14

# Health record
GET vg:health:14

# Drop the auto-transport probe cache for one camera
DEL vg:probe:197.155.67.50:554 # internal cache TTL = 5 min anyway
```

Tasks

```
# Trigger any beat task immediately
docker compose exec worker python -c "
from app.tasks.staff_classifier import classify_today
classify_today.delay()"

# List registered tasks
docker compose exec worker celery -A app.tasks.celery_app inspect registered
```

13. Glossary

Track signature	Per-camera tracker handle in the form 'cam{camera_id};tr{track_id}'. Same person across two cameras gets two signatures unless ReID is enabled (not yet implemented).
Session window	The store's business-hours bounds for today, computed from business_hours_json. Dashboards clamp aggregates to this window so cleaner-shift noise doesn't pollute KPIs.
Staff classification	A track is flagged 'staff' once it accumulates >10 min of dwell inside a counter-tagged zone today. Analytics LEFT-JOIN visitor_tracks → staff_tracks and exclude staff signatures.
RAG status	Red / Amber / Green per-store status on the chain dashboard. Red = open AND (no cameras live OR critical alert in last hour). Amber = open AND (staff < 70% OR some cameras offline). Green = otherwise.
Tunnel mode	FFmpeg's -rtsp_transport http flag. RTSP is negotiated through the HTTP port instead of opening a separate 554 connection. URL syntax is unchanged.
Detector catalog	Single source-of-truth dict (app/api/detector_catalog.py) mapping every detector_type to its status, default thresholds, and required zone purpose.
Auto-failover	Per-camera + chain-wide endpoint that probes RTSP/TCP → RTSP/tunnel → HTTP-snapshot and persists whichever combination produces video. Operators rarely need to configure transport manually.
Heatmap window	vg:heatmap:hour:{id} / day:{id} / week:{id} — three independent per-camera grids with their own reset boundaries. Dashboards request the right key via ?window=hour day week.
Briefing	Daily 08:00 store-local + Monday 07:00 chain WhatsApp summary, delivered via Twilio. Dedup by Redis day-key / iso-week-key.
Visit quality score	$\text{avg_dwell_minutes} \times (\text{zones_visited} / \text{total_zones}) \times 100$. Bounded 0-100ish; surfaced on the Daily performance scorecard's intelligence band.

Document version

Generated by scripts/build_docs_pdf.py. Re-run after material changes; commit the regenerated PDF alongside the source changes so non-technical readers always have the current document.